МДК 02.02 · ПРОФЕССИОНАЛЬНЫЙ МОДУЛЬ ПМ.02

Введение в инструментальные средства разработки ПО

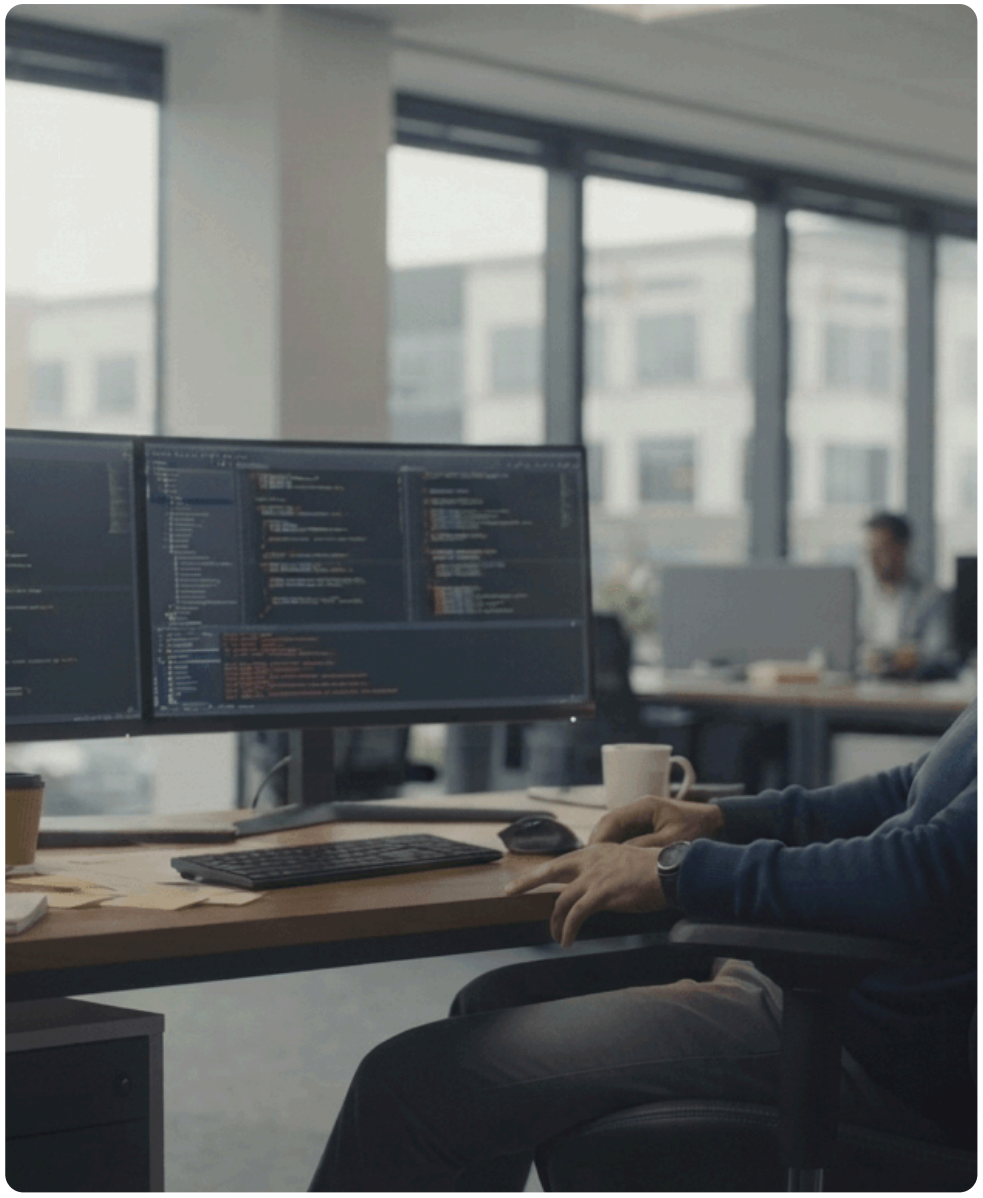
Занятие № 1 · Лекция

На этой лекции разберёмся, из каких инструментов состоит современный процесс создания программного обеспечения, зачем они нужны и как связаны между собой. Увидим путь кода от первой строки до работающего приложения и познакомимся с практиками, которые сегодня задают стандарт индустрии.

Специальность 09.02.07 „Информационные системы и программирование“ · Раздел 1 · Неделя 1 · 2 курс

ФОРМИРУЕМЫЕ КОМПЕТЕНЦИИ

ОК 01 ОК 02 ПК 2.1



ЧАСТЬ 1 ИЗ 2

Инструментальная инфраструктура разработки

Начнём с фундамента: разберём само понятие инструментальных средств и проследим путь кода через пайплайн разработки. Затем увидим, как инструменты поддерживают каждый этап жизненного цикла программного продукта.

- Понятие ИСР ПО
- Пайплайн разработки
- Жизненный цикл ПО

Проблемный вопрос: как большому коллективу разработчиков работать над одним проектом без хаоса — кто, что и когда меняет в коде?

Понятие инструментальных средств разработки ПО

Базовое определение и классификация по назначению

Инструментальные средства разработки ПО — совокупность программных инструментов, библиотек, фреймворков и сервисов, применяемых на всех этапах жизненного цикла программного продукта: от анализа требований до сопровождения готового решения. Иными словами, это весь рабочий инструментарий разработчика — всё то, чем он создаёт, проверяет и запускает программы.

Системы контроля версий

Фиксируют историю изменений кода и позволяют команде работать над проектом параллельно (Git).

Интегрированные среды разработки (IDE)

Объединяют редактор кода, отладчик и запуск проекта в одном окне (VS Code, IntelliJ IDEA).

Сборка и управление зависимостями

Собирают проект из исходного кода и подтягивают сторонние библиотеки (Maven, npm).

Инструменты тестирования

Автоматически проверяют, что код работает и не сломался после изменений (Jest, PyTest).

Контейнеризация

Упаковывают приложение вместе с окружением в переносимый контейнер (Docker).

Непрерывная интеграция и доставка (CI/CD)

Автоматически собирают, проверяют и поставляют каждую новую версию (GitHub Actions).

Мониторинг и логирование

Следят за работающим приложением и собирают данные об ошибках (Grafana, ELK).

AI-ассистенты

Дополняют код, находят ошибки и объясняют незнакомые решения (Copilot).

Курс изучает не отдельные инструменты поодиночке, а **конвейер разработки** — от инициации проекта до доставки конечному пользователю. Каждая категория займёт своё место на этом конвейере.

Пайплайн разработки

Базовая единица работы команды: инструменты тесно интегрированы — результат одного шага автоматически запускает следующий

1
Commit в Git
Разработчик записывает изменения кода в репозиторий.

→

2
Запуск CI/CD
Пайплайн стартует автоматически, без участия человека.

→

3
Сборка Docker-образа
Приложение упаковывается вместе со всеми зависимостями.

→

4
Тесты и линтеры
Автопроверка: код работает и соответствует стандартам.

→

5
Развёртывание в staging
Версия выкладывается в тестовое окружение для проверки.

→

6
Уведомление команды
Результат прогона приходит в общий мессенджер.

зелёная отметка — шаг выполняется без участия человека

> *Git-репозиторий автоматически запускает CI/CD-пайплайн, который собирает Docker-образ, прогоняет тесты, разворачивает приложение в staging-окружении и уведомляет команду через мессенджер.*

Этот конвейер называется **„пайплайном разработки“**. Сбой на любом этапе мгновенно становится известен всей команде — ошибки не накапливаются, а исправляются сразу, пока изменение свежо в памяти.

Жизненный цикл ПО (SDLC) и инструменты

Software Development Life Cycle: каждому этапу — свой инструментарий

1

Анализ требований

Определяют, что именно должен делать продукт и какие проблемы решать.

`трекинг задач` `Confluence`

2

Проектирование

Продумывают устройство системы и фиксируют его в наглядных схемах.

`UML` `PlantUML` `Mermaid`

3

Кодирование

Пишут код, проверяя его прямо в среде разработки.

`IDE` `AI-ассистенты`

4

Тестирование

Проверяют, что программа работает правильно и не содержит ошибок.

`Jest` `PyTest`

5

Развёртывание

Упаковывают и запускают приложение на реальных серверах.

`Docker` `Kubernetes` `CI/CD`

6

Сопровождение

Следят за работой продукта, исправляют ошибки и развивают функциональность.

`Prometheus` `Grafana` `ELK` `Jira`



Современный разработчик должен не просто знать отдельные инструменты, но и понимать, как они взаимодействуют в рамках единого процесса — **от коммита до развёртывания**.



Такой подход называется **DevOps** — и стал стандартом индустрии.

ЧАСТЬ 2 ИЗ 2

Современные тенденции 2024–2026

02

Как сегодня разрабатывают ПО в продуктовых и сервисных компаниях

Во второй части рассмотрим практики, которые изменили индустрию за последние годы: культуру DevOps, подход GitOps и новую роль искусственного интеллекта в работе разработчика.

- DevOps
- GitOps
- AI-driven development

+ DevSecOps — безопасность на каждом этапе пайплайна

Изучаемый в курсе инструментарий соответствует **требованиям современной индустрии**, а не учебным представлениям десятилетней давности.

DevOps

От слов Development + Operations — разработка и эксплуатация

⇄ Dev + Ops

DevOps — культура и практика объединения разработки (Dev) и эксплуатации (Ops) с целью сокращения времени от идеи до продакшна и повышения надёжности продукта. **Смысл подхода** — убрать барьер между теми, кто создаёт программу, и теми, кто обеспечивает её работу у пользователей.

Ключевые практики

CI/CD

Continuous Integration / Continuous Delivery: каждая новая версия автоматически собирается, тестируется и поставляется пользователям.

IaC

Infrastructure as Code: серверы и инфраструктура описываются кодом и хранятся в репозитории, а не настраиваются вручную.

Микросервисы

Приложение собирается из независимых небольших сервисов, которые можно разрабатывать и обновлять по отдельности.

Контейнеризация

Каждый сервис работает в изолированном переносимом окружении на базе Docker — одинаково на любой машине.

В 2024–2025 годах **DevOps — стандарт индустрии** и базовый навык любого разработчика, претендующего на работу в продуктовой или сервисной компании.

GitOps

Развитие DevOps-практики

GitOps — практика, при которой конфигурация инфраструктуры и параметры развёртывания хранятся в Git-репозитории как единый источник истины. Желаемое состояние системы описано кодом — и это описание не расходится с реальностью.

Как это работает

1. Инфраструктура описана декларативно: в репозитории хранится описание того, какой система должна быть.
2. Любое изменение проходит через pull request и попадает в общую историю так же, как изменение кода приложения.
3. Автоматическая система постоянно приводит реальное состояние инфраструктуры к описанному в Git.

ПОПУЛЯРНЫЕ ИНСТРУМЕНТЫ `ArgoCD` `Flux` `Terraform`

Что даёт команде

Аудируемость

Вся история изменений инфраструктуры сохранена в Git: видно, кто, что и когда менял.

Откаты

Вернуть предыдущее состояние можно одним revert-коммитом — без ручных действий на серверах.

Командное обсуждение

Изменения инфраструктуры проходят код-ревью, как и обычный код приложения.

Изменение инфраструктуры = изменение кода, а значит — к ней применимы те же правила дисциплины, что и к приложению.

AI-driven development

Новейшая парадигма разработки 2024–2026

> **AI-driven development** — парадигма, в которой AI-ассистенты становятся полноценным участником процесса разработки: пишут код вместе с человеком, проверяют его и объясняют решения.

Что делают AI-ассистенты

- **Дописывают рутинный код** — повторяющиеся фрагменты генерируются автоматически
- **Генерируют unit-тесты** — создают проверки для написанного кода
- **Объясняют незнакомый код** — комментируют, что делает чужой фрагмент программы
- **Проводят первичный code review** — находят явные проблемы до проверки человеком

`GitHub Copilot` `Cursor` `ChatGPT` `Claude`

> 50 % профессиональных разработчиков регулярно используют AI-ассистентов — **GitHub Octoverse 2024**



Новые вопросы

- Авторское право на сгенерированный код
- Безопасность предлагаемых решений
- Необходимость верификации каждой подсказки
- Ответственность за результат работы

Контрольные вопросы

Фронтальная беседа по ключевым понятиям занятия — проверим, как усвоен материал

1. Что такое инструментальные средства разработки ПО? Приведите классификацию по этапам жизненного цикла.
2. В чём отличие DevOps от классической модели разработки waterfall?
3. Что такое GitOps и какое преимущество он даёт команде разработки?
4. Назовите основные инструменты современного разработчика. Какие из них вы уже использовали?
5. Какие новые вопросы и проблемы появляются при использовании AI-ассистентов в разработке?
6. Опишите типичный конвейер разработки: от коммита до развёртывания в staging.
7. Какие этапы включает жизненный цикл ПО? Какие инструменты применяются на каждом этапе?